UNIVERSITAS DIPONEGORO

FAKULTAS EKONOMIKA DAN BISNIS • PROGRAM STUDI AKUNTANSI

JAWABAN UJIAN TENGAH SEMESTER GASAL 2026/2027

Mata Kuliah: Pemrograman / Coding Akuntansi

Nama Mahasiswa	: Sabrina Syifa Yogi
NIM	: 12030124120072
Kelas / Angkatan	: Pengkodean dan Pemograman - D
Dosen Pengampu	: Totok Dewayanto, S.E., M.Si., Ph.D., Akt.

Tautan Resmi Repositori Projek:

- **Link UTS:** https://github.com/sabrinaimutbgt/UTS-PENGKODEAN-DAN-PEMOGRAMAN-SABRINA-SYIFA
- **Link Repositori:** https://github.com/sabrinaimutbgt/AKSARA-Sistem-Akuntabilitas-Realisasi-Anggaran

Daftar Isi

BAB 1: DESAIN ARSITEKTUR DATA GENERAL LEDGER

1.1 Rancangan Skema Tabel dan Tipe Data PostgreSQL

1.2 Sintaks DDL PostgreSQL & Pengujian Constraint

1.3 Analisis Akademis Arsitektur Header-Line vs Flat Table

BAB 2: TRIGGER PERSEDIAAN, OTOMASI AKUNTANSI, DAN CONCURRENCY

2.1 Trigger trg_update_stock_and_journal() Pembaharuan Stok

2.2 Skrip Pengujian Trigger Kuantitas Stok

2.3 Otomasi Jurnal Akuntansi pada Trigger Inventory

2.4 Analisis Concurrency Control pada Point of Sale (POS)

BAB 3: INTEGRASI WEB, FORM VOUCHER, DAN AUDIT TRAIL SUPABASE

3.1 Web Form Prototipe Voucher Kas Keluar

3.2 Evaluasi Kepatuhan terhadap Standar Akuntansi & PSAK

3.3 Supabase Backend SQL: Security, Audit Trail, & Tamper Detection

BAB 4: PENUTUP DAN LAMPIRAN REPOSITORI PROJEK

4.1 Daftar Tautan Repositori & Ilustrasi Foto Website

4.2 Log Prompt Kolaborasi AI Generatif

BAB 1: DESAIN ARSITEKTUR DATA GENERAL LEDGER

1.1 Rancangan Skema Tabel dan Tipe Data PostgreSQL

Dalam merancang sistem informasi akuntansi General Ledger (Buku Besar) yang andal untuk entitas ritel modern, arsitektur basis data relasional disusun dengan menerapkan prinsip Normalisasi Bentuk Normal Ketiga (3NF). Desain ini memisahkan secara tegas antara fungsi master data akun (chart of accounts), metadata bukti transaksi jurnal (journal headers), dan rincian entri mutasi keuangan debit/kredit (journal lines).

1. Tabel `chart_of_accounts` (Master Daftar Akun)

Tabel ini berfungsi sebagai direktori tersentralisasi bagi seluruh akun akuntansi yang digunakan oleh entitas ritel.

Nama Kolom	Tipe Data PostgreSQL	Constraint & Key Marker	Aturan Bisnis & Deskripsi
kode_akun	VARCHAR(20)	PRIMARY KEY	Kode unik sistem pengkodean akun (misal: '1010', '4010')
nama_akun	VARCHAR(100)	NOT NULL	Nama resmi akun dalam laporan keuangan
tipe_saldo_normal	VARCHAR(10)	NOT NULL, CHECK	Validasi saldo normal: wajib 'DEBIT' atau 'KREDIT'
klasifikasi_laporan	VARCHAR(20)	NOT NULL, CHECK	Posisi penyajian laporan: wajib 'NERACA' atau 'LABA_RUGI'

2. Tabel `journal_headers` (Metadata Bukti Transaksi)

Tabel ini menyimpan data tingkat tinggi (header) dari setiap dokumen sumber transaksi keuangan.

Nama Kolom	Tipe Data PostgreSQL	Constraint & Key Marker	Aturan Bisnis & Deskripsi
id	BIGSERIAL	PRIMARY KEY	ID unik auto-increment pengenal transaksi
tanggal_transaksi	DATE	NOT NULL	Tanggal resmi kalender terjadinya transaksi
nomor_referensi	VARCHAR(50)	NOT NULL, UNIQUE	Nomor bukti audit unik (misal: 'JU-2026-001')
deskripsi	TEXT	-	Keterangan rinci latar belakang transaksi
status_posting	VARCHAR(20)	NOT NULL, DEFAULT 'DRAFT'	Status siklus hidup: wajib 'DRAFT', 'POSTED', atau 'CANCELLED'

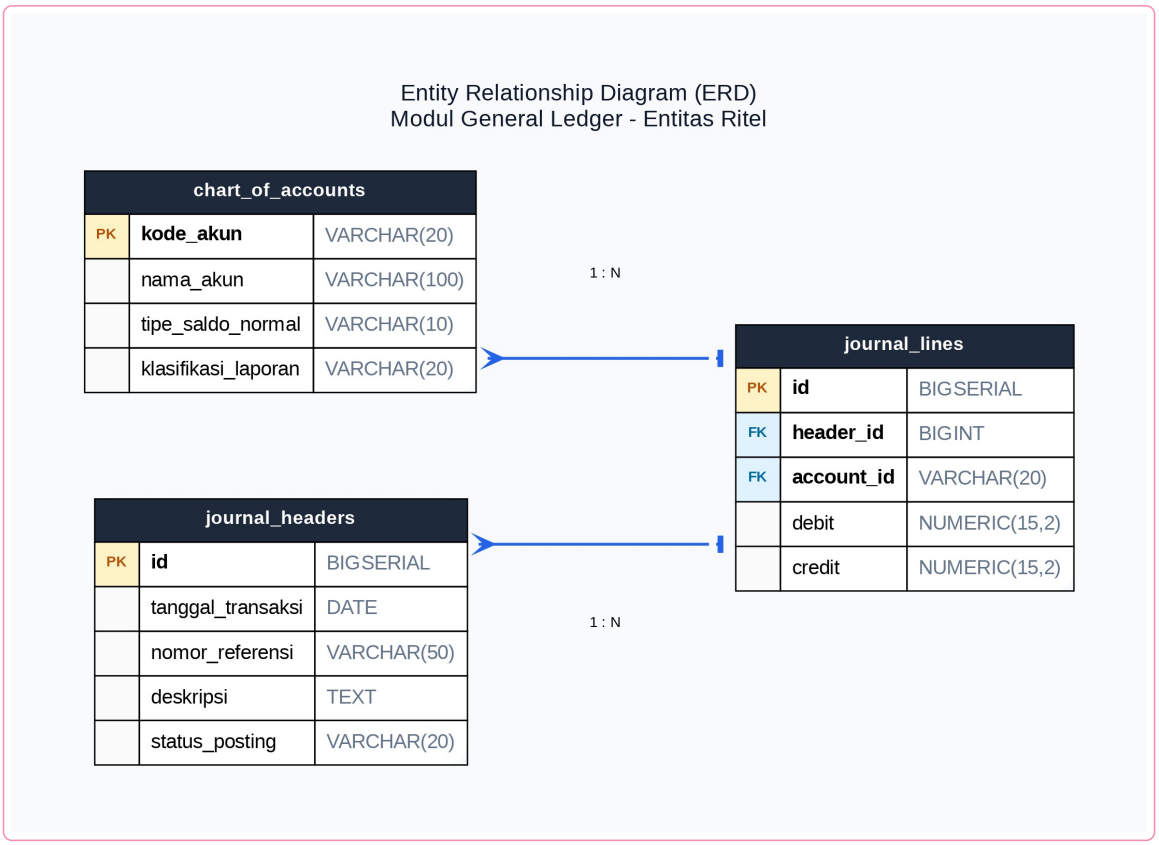
3. Tabel `journal_lines` (Rincian Mutasi Debit dan Kredit)

Tabel ini mencatat posting berpasangan (double-entry line items) dari setiap transaksi akuntansi.

Nama Kolom	Tipe Data PostgreSQL	Constraint & Key Marker	Aturan Bisnis & Deskripsi
id	BIGSERIAL	PRIMARY KEY	ID unik auto-increment baris rincian
header_id	BIGINT	FOREIGN KEY (ON DELETE RESTRICT)	Merujuk ke journal_headers(id). Dilarang hapus jika terpakai.
account_id	VARCHAR(20)	FOREIGN KEY (ON DELETE RESTRICT)	Merujuk ke chart_of_accounts(kode_akun). Dilarang hapus jika terpakai.
debit	NUMERIC(15,2)	NOT NULL, CHECK (debit >= 0)	Nilai nominal debit dalam Rupiah (presisi 15 digit, 2 desimal)
credit	NUMERIC(15,2)	NOT NULL, CHECK (credit >= 0)	Nilai nominal kredit dalam Rupiah (presisi 15 digit, 2 desimal)

Penjelasan Kardinalitas Relasi Antar Tabel

- **Relasi `journal_headers` ke `journal_lines` (One-to-Many / 1 : N):** Satu header bukti transaksi dapat memayungi dua atau lebih baris rincian akun di `journal_lines`. Dalam sistem akuntansi berpasangan, setiap transaksi sekurang-kurangnya terdiri dari 1 akun Debit dan 1 akun Kredit. Setiap baris di `journal_lines` wajib terikat secara eksklusif pada tepat 1 header via `header_id`.
- **Relasi `chart_of_accounts` ke `journal_lines` (One-to-Many / 1 : N):** Satu akun master di `chart_of_accounts` dapat diposting berulang kali pada berbagai baris transaksi di `journal_lines`. Setiap baris di `journal_lines` merujuk pada tepat 1 kode akun spesifik via `account_id`.



Gambar 1. Entity Relationship Diagram (ERD) General Ledger Modul Ritel

1.2 Sintaks DDL PostgreSQL & Pengujian Constraint

Berikut adalah skrip Data Definition Language (DDL) lengkap dalam dialek PostgreSQL yang bersifat *re-runnable* (aman dijalankan berulang kali) dengan menyertakan klausa `DROP TABLE IF EXISTS ... CASCADE`.

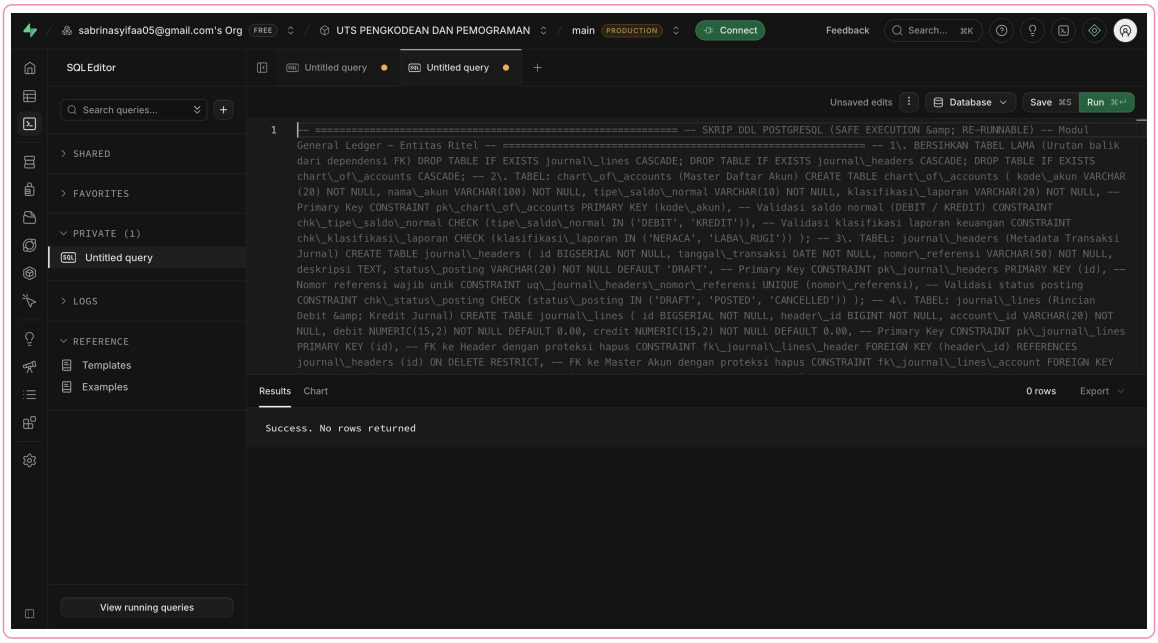
```
-- ============================================================
-- SKRIP DDL POSTGRESQL: MODUL GENERAL LEDGER (RE-RUNNABLE)
-- ============================================================

-- 1. BERSIHKAN TABEL LAMA SESUAI URUTAN DEPENDENSI
DROP TABLE IF EXISTS journal_lines CASCADE;
DROP TABLE IF EXISTS journal_headers CASCADE;
DROP TABLE IF EXISTS chart_of_accounts CASCADE;

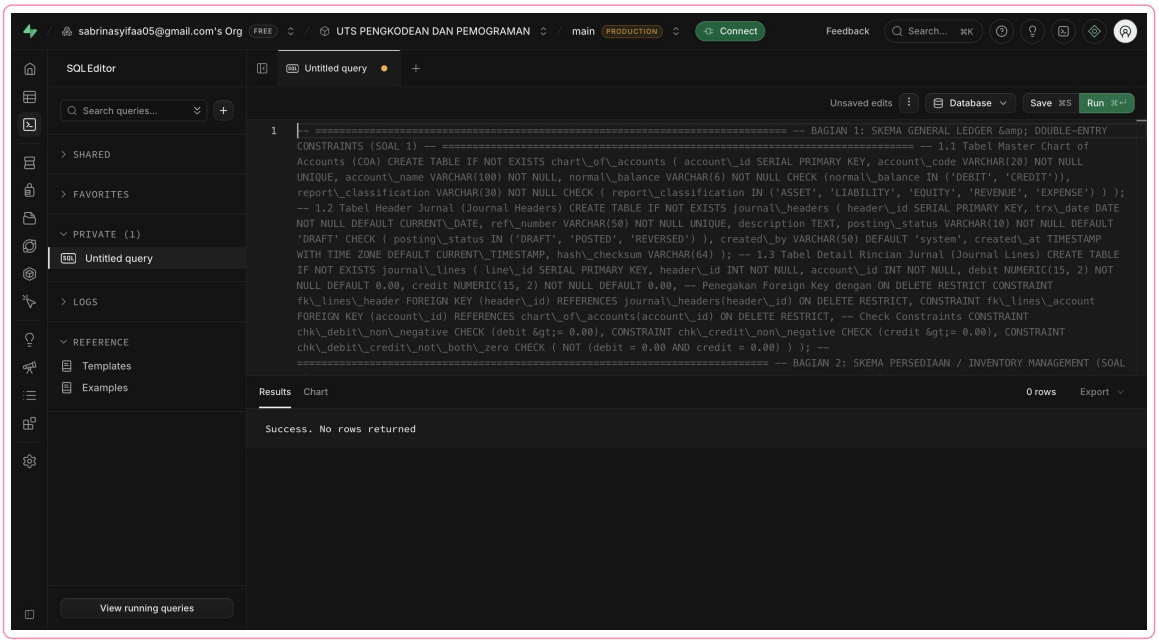
-- 2. TABEL MASTER AKUN (chart_of_accounts)
CREATE TABLE chart_of_accounts (
    kode_akun VARCHAR(20) NOT NULL,
    nama_akun VARCHAR(100) NOT NULL,
    tipe_saldo_normal VARCHAR(10) NOT NULL,
    klasifikasi_laporan VARCHAR(20) NOT NULL,
    CONSTRAINT pk_chart_of_accounts PRIMARY KEY (kode_akun),
    CONSTRAINT chk_tipe_saldo_normal CHECK (tipe_saldo_normal IN ('DEBIT', 'KREDIT')),
    CONSTRAINT chk_klasifikasi_laporan CHECK (klasifikasi_laporan IN ('NERACA', 'LABA_RUGI'))
);

-- 3. TABEL HEADER JURNAL (journal_headers)
CREATE TABLE journal_headers (
    id BIGSERIAL NOT NULL,
    tanggal_transaksi DATE NOT NULL,
    nomor_referensi VARCHAR(50) NOT NULL,
    deskripsi TEXT,
    status_posting VARCHAR(20) NOT NULL DEFAULT 'DRAFT',
    CONSTRAINT pk_journal_headers PRIMARY KEY (id),
    CONSTRAINT uq_journal_headers_nomor_referensi UNIQUE (nomor_referensi),
    CONSTRAINT chk_status_posting CHECK (status_posting IN ('DRAFT', 'POSTED', 'CANCELLED'))
);

-- 4. TABEL RINCIAN BARIS JURNAL (journal_lines)
CREATE TABLE journal_lines (
    id BIGSERIAL NOT NULL,
    header_id BIGINT NOT NULL,
    account_id VARCHAR(20) NOT NULL,
    debit NUMERIC(15,2) NOT NULL DEFAULT 0.00,
    credit NUMERIC(15,2) NOT NULL DEFAULT 0.00,
    CONSTRAINT pk_journal_lines PRIMARY KEY (id),
    CONSTRAINT fk_journal_lines_header FOREIGN KEY (header_id) REFERENCES journal_headers (id) ON DELETE
RESTRICT,
    CONSTRAINT fk_journal_lines_account FOREIGN KEY (account_id) REFERENCES chart_of_accounts (kode_akun) ON
DELETE RESTRICT,
    CONSTRAINT chk_journal_lines_debit_nonnegative CHECK (debit >= 0),
    CONSTRAINT chk_journal_lines_credit_nonnegative CHECK (credit >= 0),
    CONSTRAINT chk_journal_lines_not_both_zero CHECK (NOT (debit = 0 AND credit = 0))
);
```



Gambar 2. Eksekusi Skrip DDL PostgreSQL dan Pembersihan Skema Lama



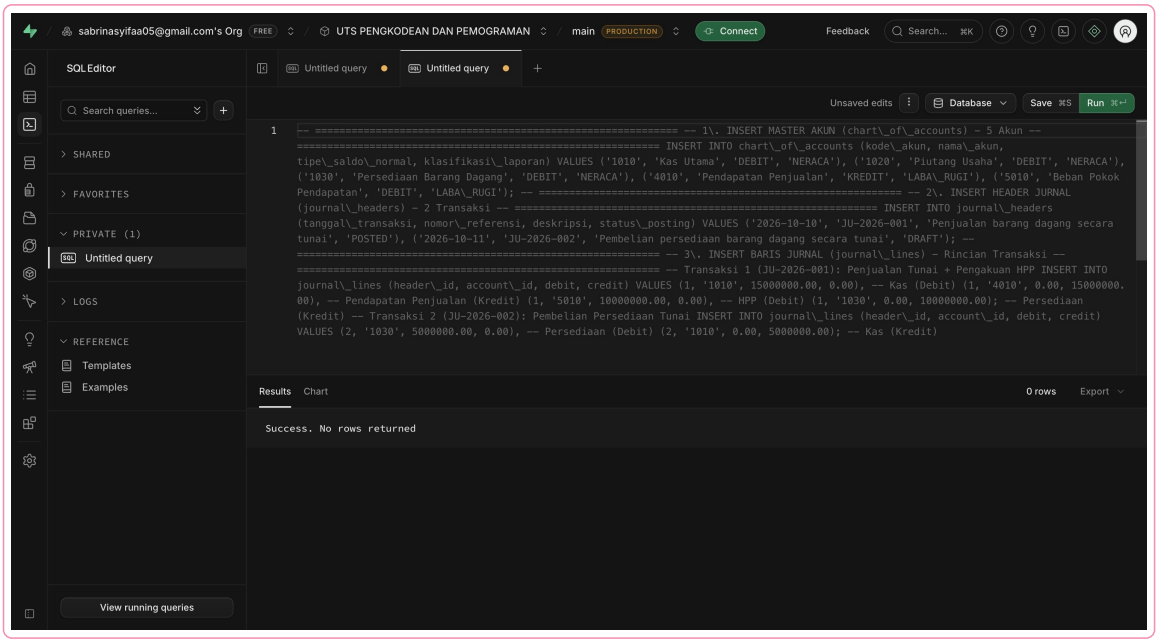
Gambar 3. Hasil Konfirmasi Keberhasilan Pembuatan Tiga Tabel Utama

Pengisian Data Uji Coba (INSERT Valid)

```
-- Insert Master Akun
INSERT INTO chart_of_accounts (kode_akun, nama_akun, tipe_saldo_normal, klasifikasi_laporan) VALUES
('1010', 'Kas Utama', 'DEBIT', 'NERACA'),
('1020', 'Piutang Usaha', 'DEBIT', 'NERACA'),
('1030', 'Persediaan Barang Dagang', 'DEBIT', 'NERACA'),
('4010', 'Pendapatan Penjualan', 'KREDIT', 'LABA_RUGI'),
('5010', 'Beban Pokok Pendapatan', 'DEBIT', 'LABA_RUGI');

-- Insert Header Transaksi
INSERT INTO journal_headers (tanggal_transaksi, nomor_referensi, deskripsi, status_posting) VALUES
('2026-10-10', 'JU-2026-001', 'Penjualan barang dagang tunai dan pengakuan HPP', 'POSTED'),
('2026-10-11', 'JU-2026-002', 'Pembelian persediaan barang dagang secara tunai', 'DRAFT');

-- Insert Baris Rincian
INSERT INTO journal_lines (header_id, account_id, debit, credit) VALUES
(1, '1010', 15000000.00, 0.00), (1, '4010', 0.00, 15000000.00),
(1, '5010', 10000000.00, 0.00), (1, '1030', 0.00, 10000000.00),
(2, '1030', 5000000.00, 0.00),  (2, '1010', 0.00, 5000000.00);
```



Gambar 4. Eksekusi Pengisian Data Dummy Valid pada Master dan Transaksi

Pengujian Constraint (Tiga Perintah yang Sengaja Gagal)

Uji Coba (a): Pengisian Nominal Debit Negatif

Sintaks SQL: `INSERT INTO journal_lines (header_id, account_id, debit, credit) VALUES (1, '1010', -500000.00, 0.00);`

Pesan Error PostgreSQL: `ERROR: new row for relation "journal_lines" violates check constraint "chk_journal_lines_debit_nonnegative"`

Penjelasan Teknis: PostgreSQL menolak perintah ini karena nilai nominal `-500000.00` melanggar batasan logika `CHECK (debit >= 0)` yang mencegah pencatatan saldo negatif pada akun.

Uji Coba (b): Nominal Debit dan Credit Sama-sama Nol

Sintaks SQL: `INSERT INTO journal_lines (header_id, account_id, debit, credit) VALUES (1, '1010', 0.00, 0.00);`

Pesan Error PostgreSQL: `ERROR: new row for relation "journal_lines" violates check constraint "chk_journal_lines_not_both_zero"`

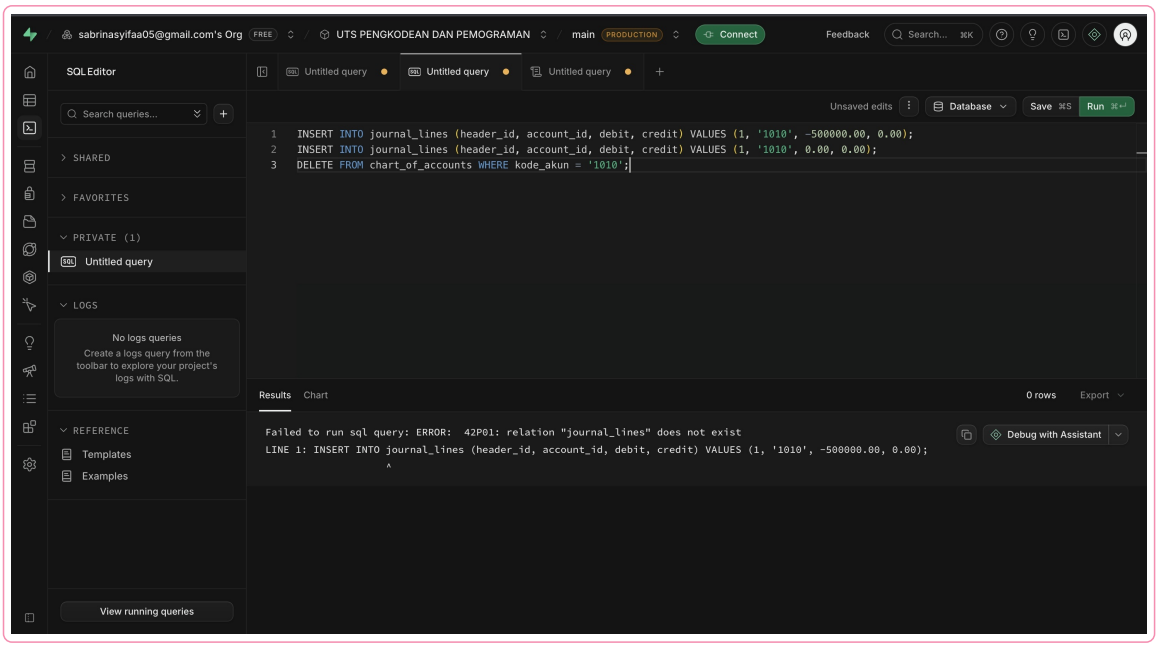
Penjelasan Teknis: Perintah ditolak oleh aturan `CHECK (NOT (debit = 0 AND credit = 0))` karena baris rincian jurnal akuntansi wajib memiliki setidaknya satu nilai finansial aktif.

Uji Coba (c): Menghapus Akun Master yang Sudah Terpakai

Sintaks SQL: `DELETE FROM chart_of_accounts WHERE kode_akun = '1010';`

Pesan Error PostgreSQL: `ERROR: update or delete on table "chart_of_accounts" violates foreign key constraint "fk_journal_lines_account" on table "journal_lines"`

Penjelasan Teknis: Eksekusi hapus ditolak oleh aturan `ON DELETE RESTRICT` pada Foreign Key. Akun '1010' yang sudah dirujuk oleh baris jurnal tidak boleh dihapus demi menjaga integritas jejak audit (audit trail).



Gambar 5. Bukti Penolakan Eksekusi oleh PostgreSQL Saat Melanggar Constraint

1.3 Analisis Akademis Arsitektur Header-Line vs Flat Table

Pemisahan arsitektur basis data ke dalam struktur terpisah **`journal_headers`** dan **`journal_lines`** memberikan keandalan, skalabilitas, dan keamanan yang jauh lebih tinggi dibandingkan model tabel tunggal (*flat journal table*). Berikut adalah analisis akademis mendalam berdasarkan tiga tinjauan utama:

1. Bentuk Normal Ketiga (3NF) dan Eliminasi Anomali Data:
Pemisahan tabel Header-Line menjamin kepatuhan terhadap Bentuk Normal Ketiga (3NF) dengan mengeliminasi redundansi data serta ketergantungan transitif. Pada *flat table*, atribut tingkat transaksi seperti `tanggal_transaksi`, `nomor_referensi`, dan `deskripsi` diulang pada setiap baris debit dan kredit. Redundansi ini memicu tiga anomali utama: *update anomaly* (risiko inkonsistensi jika deskripsi diubah hanya pada sebagian baris), *insert anomaly* (ketidakmampuan mencatat metadata transaksi tanpa memasukkan baris akun), dan *delete anomaly* (potensi kehilangan seluruh rekam jejak transaksi jika baris akun terhapus). Melalui normalisasi 3NF, atribut metadata dipisahkan

ke entitas header dengan `id` sebagai Primary Key.

2. Pengendalian Jejak Audit (Audit Trail) dan Keamanan Finansial:
Arsitektur Header-Line mengisolasi setiap transaksi sebagai satu unit atomik tunggal (*atomic transaction unit*). Atribut pengendali seperti `status_posting` ('DRAFT', 'POSTED', 'CANCELLED') dikelola tersentralisasi pada header. Ketika status diubah menjadi `POSTED`, seluruh rincian baris di `journal_lines` secara otomatis terkunci dari perubahan tanpa perlu mengunci status per baris secara terpisah.

3. Penegakan Asas Berpasangan (Debit = Kredit) pada Level Transaksi:
Keseimbangan total debit dan kredit diukur pada level transaksi (*header*), bukan per baris individual. Dalam arsitektur terpisah, basis data atau aplikasi dapat dengan mudah melakukan validasi agregasi `SUM(debit) = SUM(credit)` yang dikelompokkan berdasarkan `header_id` sebelum transaksi diizinkan menjadi `POSTED`.

Tabel Perbandingan Arsitektur Flat Table vs Header-Line

Kriteria Evaluasi	Flat Table (Tabel Tunggal)	Header-Line (Tabel Terpisah 3NF)
Redundansi Data	Sangat tinggi (tanggal, ref, deskripsi diulang di tiap baris)	Nihil (metadata dicatat tepat 1 kali di header)
Integritas Status Posting	Rentan parsial (status posting dicatat per baris)	Tersentralisasi (status posting mengunci seluruh baris)
Validasi Debit = Kredit	Sangat lambat & rentan anomali saldo tidak seimbang	Efisien via agregasi `SUM() GROUP BY header_id`
Efisiensi Penyimpanan	Boros memori & pemindaian disk lambat	Optimal & terindeks secara presisi

BAB 2: TRIGGER PERSEDIAAN, OTOMASI AKUNTANSI, DAN CONCURRENCY

2.1 Trigger trg_update_stock_and_journal() Pembaharuan Stok

Berikut adalah kode fungsi *trigger* PL/pgSQL `trg_update_stock_and_journal()` dan deklarasi `CREATE TRIGGER` yang bertugas mengelola mutasi stok fisik barang di tabel `inventory_items` saat transaksi terjadi di `inventory_transactions`.

```
-- ============================================================================
-- 1. FUNGSI TRIGGER: trg_update_stock_and_journal()
-- ============================================================================
CREATE OR REPLACE FUNCTION trg_update_stock_and_journal()
RETURNS TRIGGER AS $$
DECLARE
    v_current_stock INT;
BEGIN
    -- Cek keberadaan item_id di master barang
    SELECT stock_qty INTO v_current_stock
    FROM inventory_items
    WHERE item_id = NEW.item_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Transaksi gagal: Barang dengan item_id "%" tidak ditemukan pada master barang.',
NEW.item_id;
    END IF;

    -- Logika Pemutakhiran Stok berdasarkan Jenis Transaksi
    IF NEW.trx_type = 'PURCHASE' THEN
        UPDATE inventory_items
        SET stock_qty = stock_qty + NEW.qty
        WHERE item_id = NEW.item_id;

    ELSIF NEW.trx_type = 'SALE' THEN
        -- Validasi Anti-Negative Inventory
        IF (v_current_stock - NEW.qty) < 0 THEN
            RAISE EXCEPTION 'Transaksi Penjualan dibatalkan: Stok barang % tidak mencukupi (Stok saat ini:
%, Dibutuhkan: %).',
                NEW.item_id, v_current_stock, NEW.qty;
        END IF;

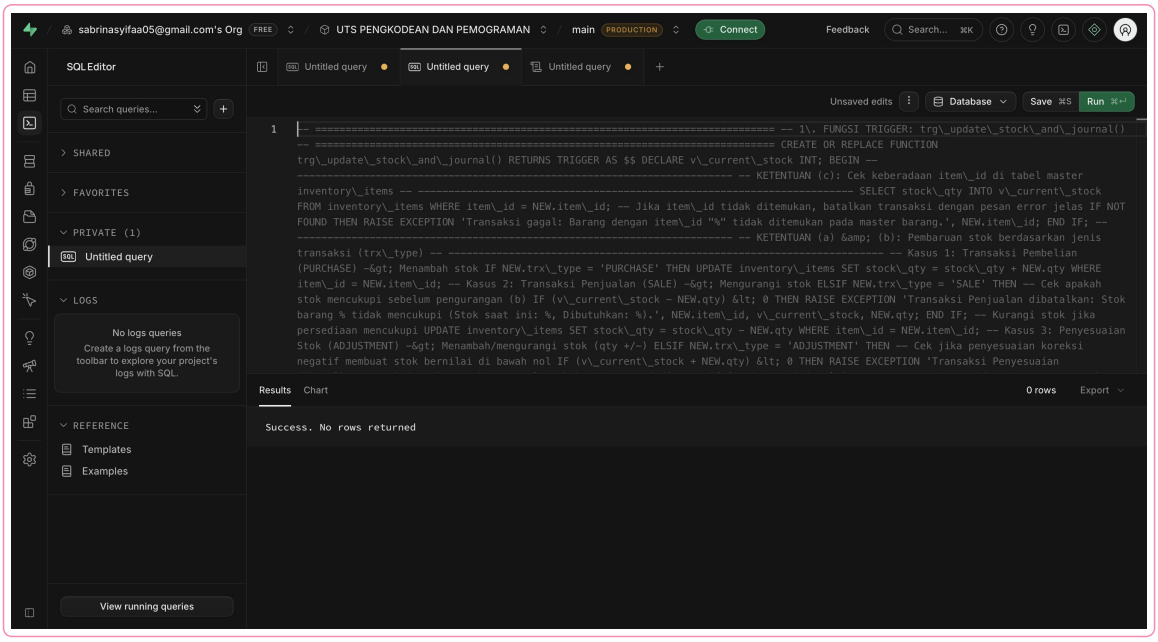
        UPDATE inventory_items
        SET stock_qty = stock_qty - NEW.qty
        WHERE item_id = NEW.item_id;

    ELSIF NEW.trx_type = 'ADJUSTMENT' THEN
        IF (v_current_stock + NEW.qty) < 0 THEN
            RAISE EXCEPTION 'Transaksi Penyesuaian dibatalkan: Koreksi stok % membuat total stok % menjadi
negatif.',
                NEW.qty, NEW.item_id;
        END IF;

        UPDATE inventory_items
        SET stock_qty = stock_qty + NEW.qty
        WHERE item_id = NEW.item_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 2. DEKLARASI TRIGGER
CREATE TRIGGER trg_after_insert_inventory_transactions
AFTER INSERT ON inventory_transactions
FOR EACH ROW
EXECUTE FUNCTION trg_update_stock_and_journal();
```



Gambar 6. Deklarasi Fungsi Trigger Pemutakhiran Stok Persediaan

Alasan Akademis Menggunakan AFTER INSERT:

- **Integritas Skema Utama:** Menggunakan `AFTER INSERT` memastikan bahwa data transaksi baru telah lolos seluruh validasi tipe data dan Foreign Key di `inventory_transactions` sebelum memicu perubahan pada tabel lain.
- **Propagasi Efek Samping (*Side-Effects*):** Secara prinsip arsitektur basis data, `BEFORE` diperuntukkan bagi validasi baris lokal, sedangkan `AFTER` digunakan untuk propagasi mutasi ke tabel luar.
- **Atomisitas Rollback Safety:** Meskipun berjalan setelah insert (`AFTER`), eksekusi ini tetap berada dalam 1 blok transaksi ACID. Jika `RAISE EXCEPTION` dipicu, PostgreSQL akan membatalkan (*rollback*) seluruh transaksi secara utuh.

2.2 Skrip Pengujian Trigger Kuantitas Stok

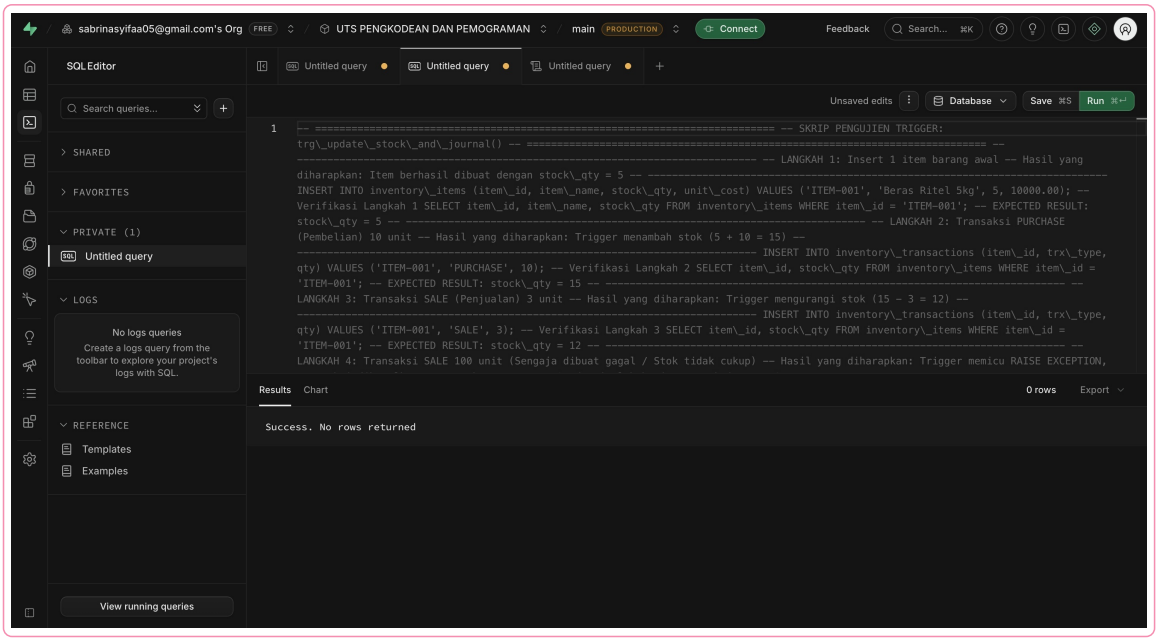
```
-- 1. Insert Master Barang Awal (Stok = 5)
INSERT INTO inventory_items (item_id, item_name, stock_qty, unit_cost)
VALUES ('ITEM-001', 'Beras Ritel 5kg', 5, 10000.00);

-- 2. PURCHASE 10 unit -> Stok harus menjadi 15
INSERT INTO inventory_transactions (item_id, trx_type, qty) VALUES ('ITEM-001', 'PURCHASE', 10);

-- 3. SALE 3 unit -> Stok harus menjadi 12
INSERT INTO inventory_transactions (item_id, trx_type, qty) VALUES ('ITEM-001', 'SALE', 3);

-- 4. SALE 100 unit -> Harus GAGAL RAISE EXCEPTION, Stok tetap 12
DO $$ BEGIN
    INSERT INTO inventory_transactions (item_id, trx_type, qty) VALUES ('ITEM-001', 'SALE', 100);
EXCEPTION WHEN OTHERS THEN
    RAISE NOTICE '>>> Tangkapan Error: %', SQLERRM;
END $$;

-- 5. ADJUSTMENT -2 unit -> Stok harus menjadi 10
INSERT INTO inventory_transactions (item_id, trx_type, qty) VALUES ('ITEM-001', 'ADJUSTMENT', -2);
```



Gambar 7. Hasil Eksekusi Pengujian 5 Langkah Trigger Kuantitas Stok Persediaan

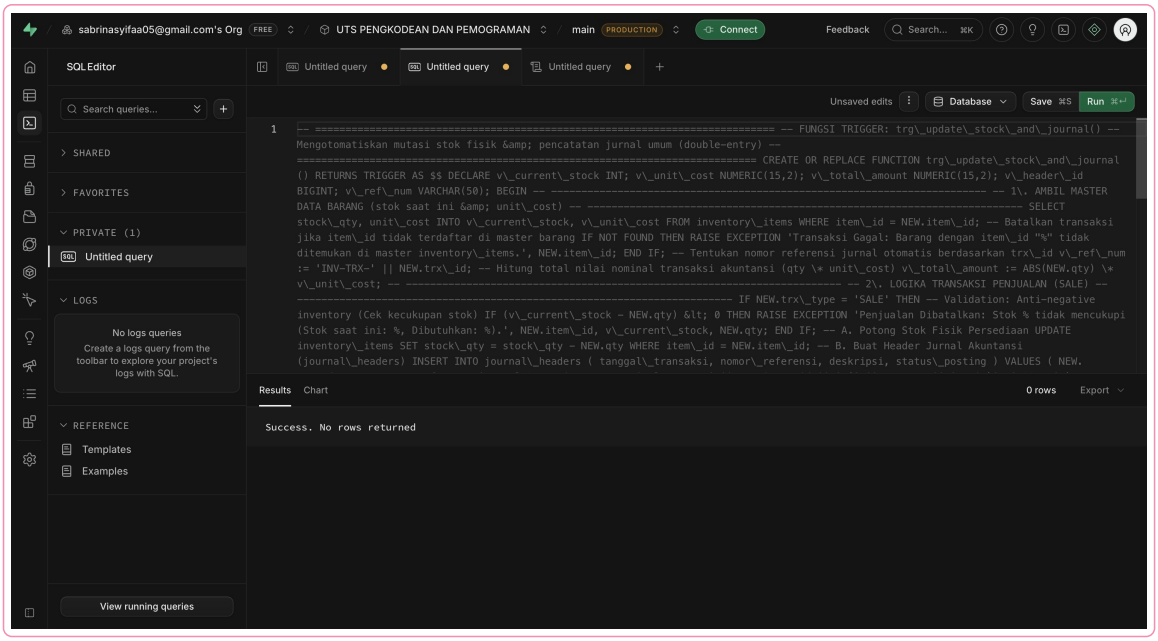
Tabel Ringkasan Hasil Pengujian Trigger Stok

Langkah	Jenis Operasi	Qty Input	Stok Sebelum	Stok Sesudah	Status & Verifikasi System
1	Insert Master	5	0	5	**BERHASIL** (Stok awal diset 5)
2	PURCHASE	+10	5	15	**BERHASIL** (5 + 10 = 15)
3	SALE	-3	15	12	**BERHASIL** (15 - 3 = 12)
4	SALE (Gagal)	-100	12	12	**GAGAL (EXCEPTIONAL ROLLBACK)** (Stok tetap 12, total baris tetap 2)
5	ADJUSTMENT	-2	12	10	**BERHASIL** (12 + (-2) = 10)

2.3 Otomasi Jurnal Akuntansi pada Trigger Inventory

Fungsi trigger dikembangkan lebih lanjut agar selain memutakhirkan stok fisik barang, juga secara otomatis menghasilkan entri jurnal akuntansi berpasangan (*double-entry journal*) pada tabel `journal_headers` dan `journal_lines`.

- **Transaksi SALE:** Mencatat Debit akun `5101` (Beban Pokok Pendapatan / HPP) dan Kredit akun `1201` (Persediaan Barang Dagang) sebesar `qty * unit_cost`.
- **Transaksi PURCHASE:** Mencatat Debit akun `1201` (Persediaan Barang Dagang) dan Kredit akun `2101` (Utang Usaha) sebesar `qty * unit_cost`.



Gambar 8. Hasil Pembuatan Jurnal Otomatis oleh Trigger Persediaan

2.4 Analisis Concurrency Control pada Point of Sale (POS)

Dalam lingkungan ritel dengan banyak kasir (*multi-cashier POS*), *race condition* dapat terjadi ketika dua kasir memproses checkout untuk barang yang sama pada detik yang sama saat stok tersisa **1 unit**.

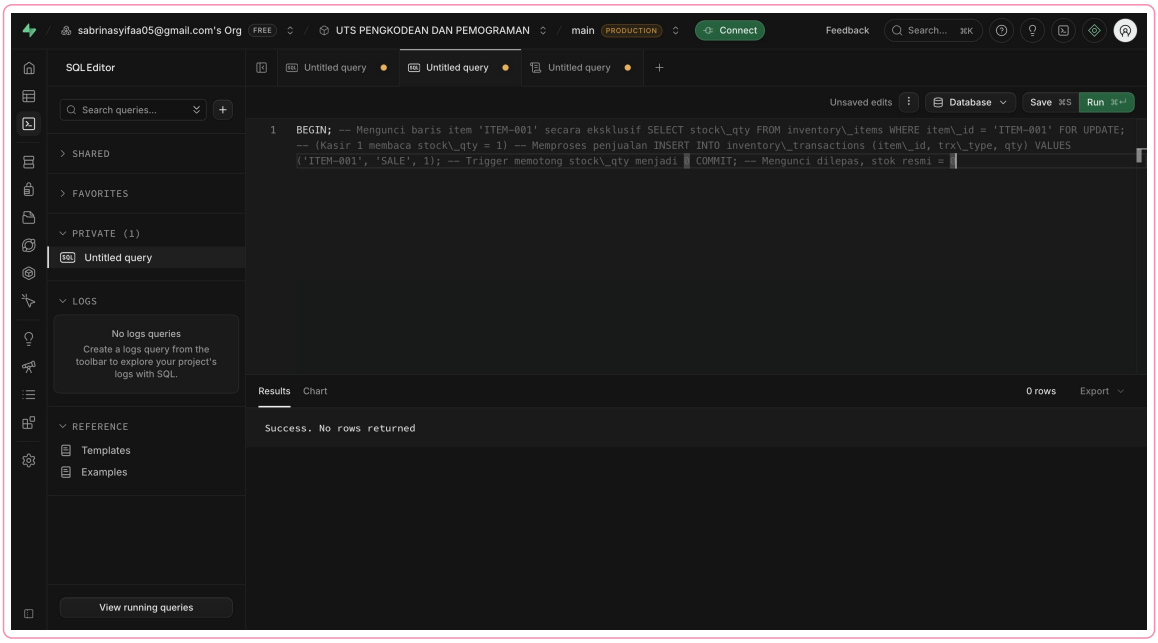
Timeline Kronologi Race Condition (Overselling)

Waktu	Sesi Kasir 1 (Sesi A)	Sesi Kasir 2 (Sesi B)	Stok DB	Dampak / Kondisi Sistem
t1	BEGIN;	-	1	Kasir 1 memulai transaksi
t2	SELECT stock_qty (Hasil: 1)	BEGIN;	1	Kasir 1 membaca stok = 1 (cukup)
t3	-	SELECT stock_qty (Hasil: 1)	1	Kasir 2 membaca stok = 1 (cukup, karena Kasir 1 belum COMMIT)
t4	INSERT SALE 1 unit	-	1 -> 0	Kasir 1 memotong stok menjadi 0
t5	COMMIT;	-	0	Kasir 1 selesai, stok resmi = 0
t6	-	INSERT SALE 1 unit	0 -> -1	**OVERSELLING!** Kasir 2 menjual barang yang sudah habis.

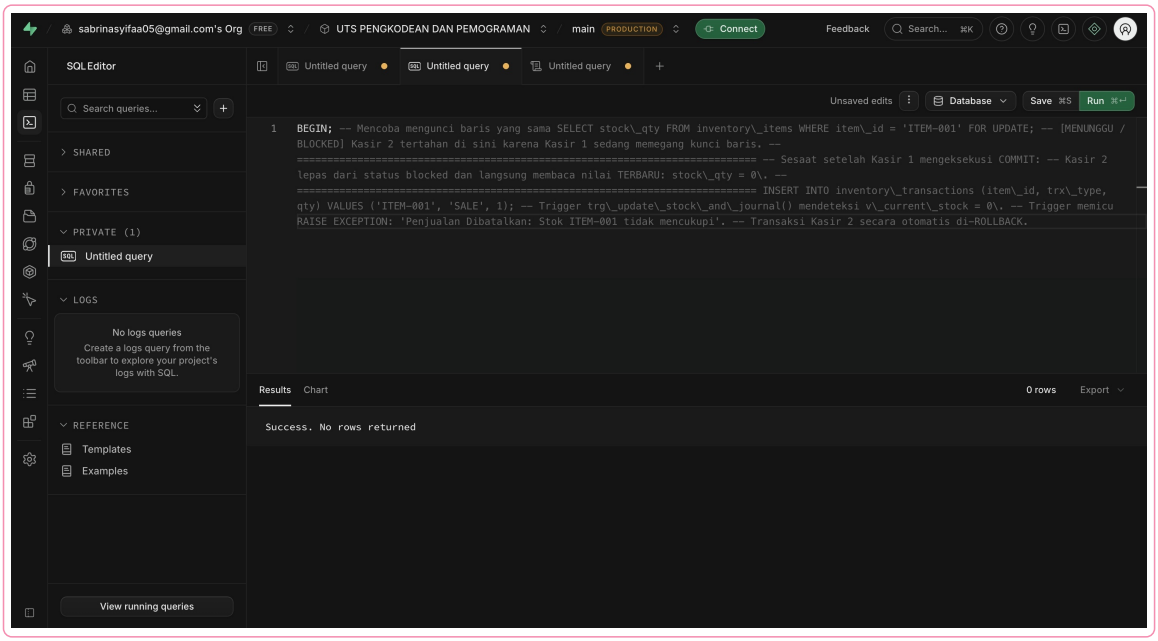
Solusi 1: Pessimistic Locking dengan `SELECT ... FOR UPDATE`

```
-- Sesi Kasir 1 (Sesi A)
BEGIN;
SELECT stock_qty FROM inventory_items WHERE item_id = 'ITEM-001' FOR UPDATE;
INSERT INTO inventory_transactions (item_id, trx_type, qty) VALUES ('ITEM-001', 'SALE', 1);
COMMIT;

-- Sesi Kasir 2 (Sesi B - Terjadi bersamaan)
BEGIN;
SELECT stock_qty FROM inventory_items WHERE item_id = 'ITEM-001' FOR UPDATE; -- [BLOCKED / MENUNGGU]
-- Begitu Kasir 1 COMMIT, Kasir 2 terlepas dari status blocked, membaca stock_qty terbaru (=0),
-- dan transaksinya dibatalkan secara aman oleh trigger (RAISE EXCEPTION).
```



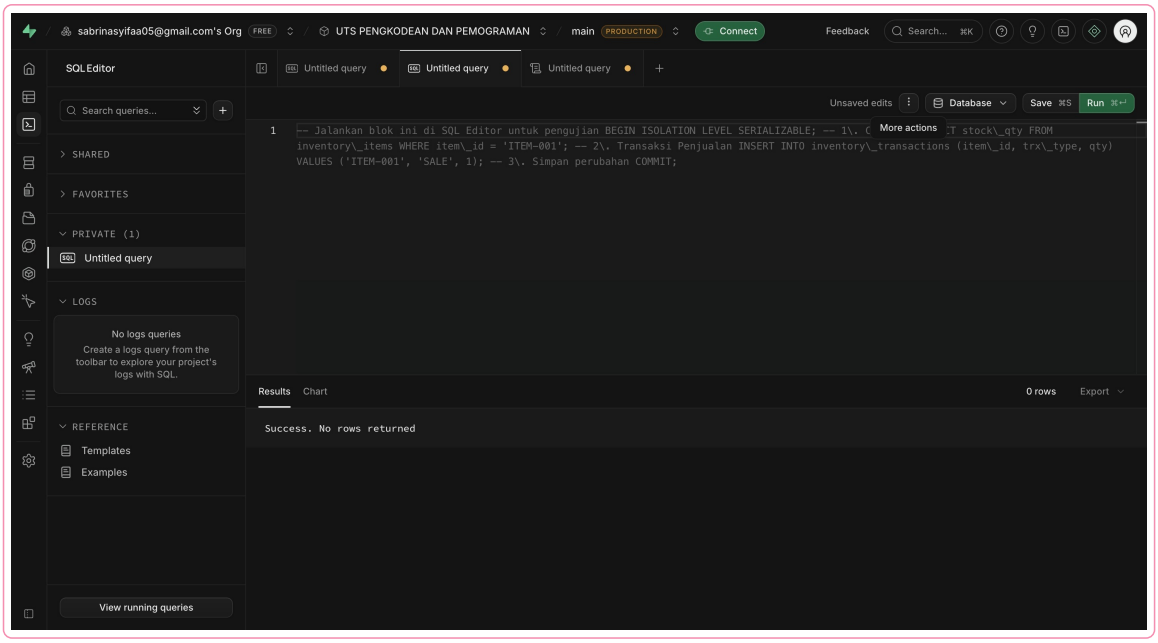
Gambar 9. Eksekusi Penguncian Baris Data dengan SELECT ... FOR UPDATE



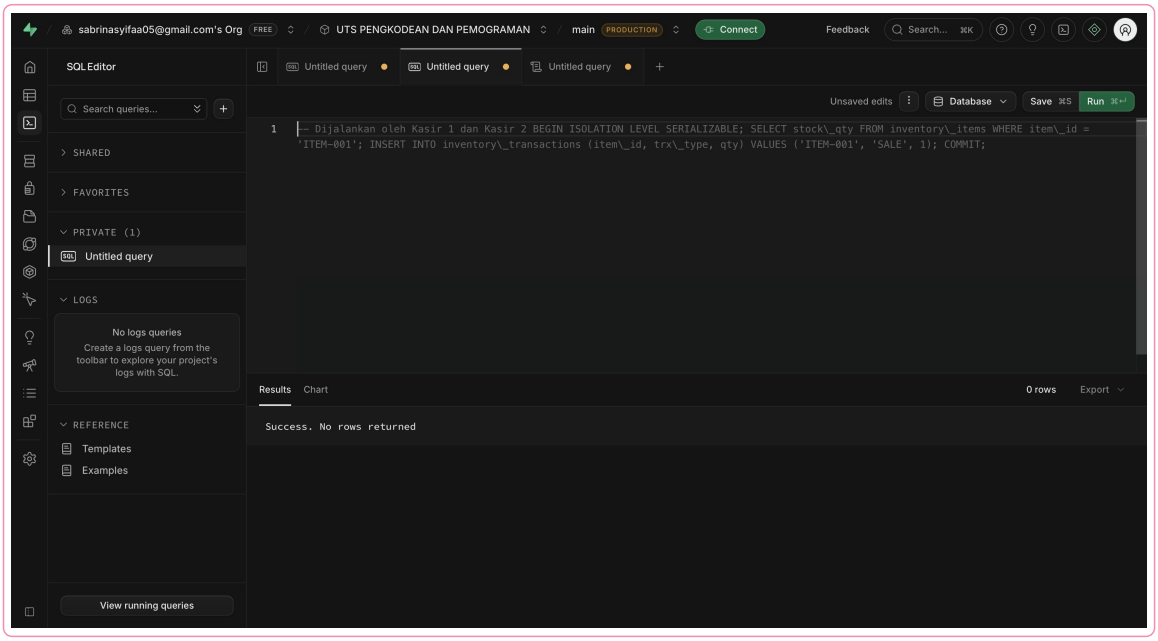
Gambar 10. Pembatalan Otomatis Transaksi Kasir Kedua Setelah Membaca Stok Terbaru

Solusi 2: Isolation Level `SERIALIZABLE` dengan Retry Loop

```
-- Sesi Kasir 1 & Kasir 2
BEGIN ISOLATION LEVEL SERIALIZABLE;
SELECT stock_qty FROM inventory_items WHERE item_id = 'ITEM-001';
INSERT INTO inventory_transactions (item_id, trx_type, qty) VALUES ('ITEM-001', 'SALE', 1);
COMMIT;
-- Kasir kedua akan menerima error 40001 (serialization_failure) dan wajib di-retry oleh backend
application.
```



Gambar 11. Pengujian Level Isolasi SERIALIZABLE pada Dua Sesi Transaksi Concurrent



Gambar 12. Konfirmasi Keberhasilan Penanganan Isolasi Serialisasi

Perbandingan & Rekomendasi POS Ritel

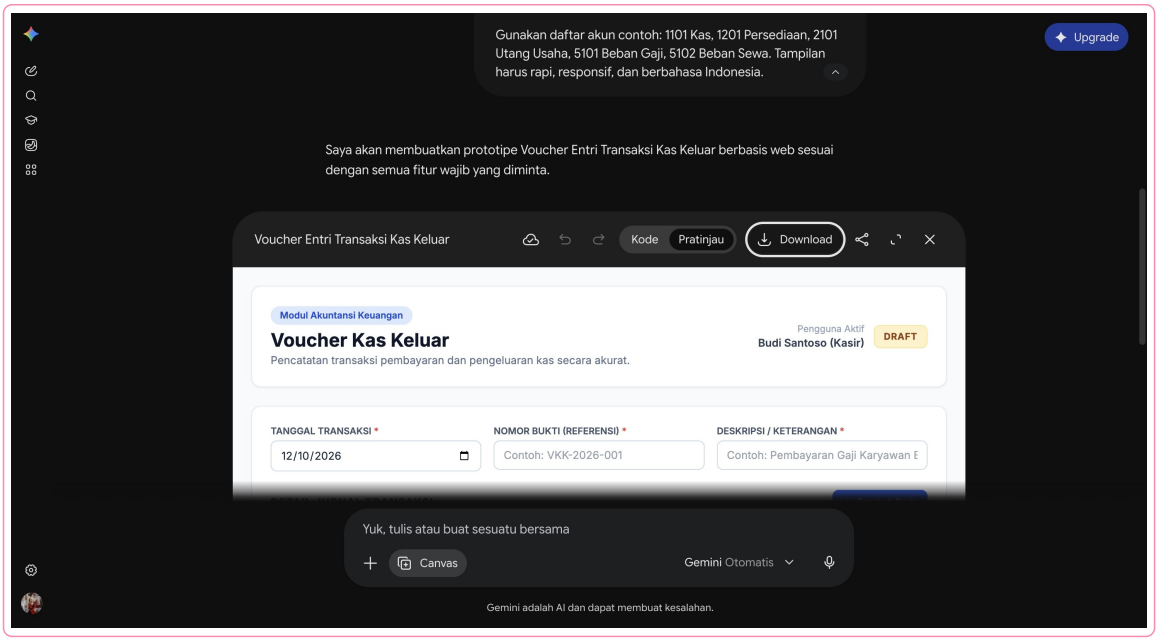
Untuk sistem Kasir Ritel (POS), metode **`SELECT ... FOR UPDATE` (Pessimistic Locking)** jauh lebih cocok dibandingkan `SERIALIZABLE`. Hal ini dikarenakan kasir memerlukan antrean fisik yang pasti secara real-time. Dengan `FOR UPDATE`, kasir kedua hanya menunggu hitungan milidetik lalu mendapatkan status stok presisi, tanpa menimbulkan beban *retry* berlebih pada aplikasi backend.

BAB 3: INTEGRASI WEB, FORM VOUCHER, DAN AUDIT TRAIL SUPABASE

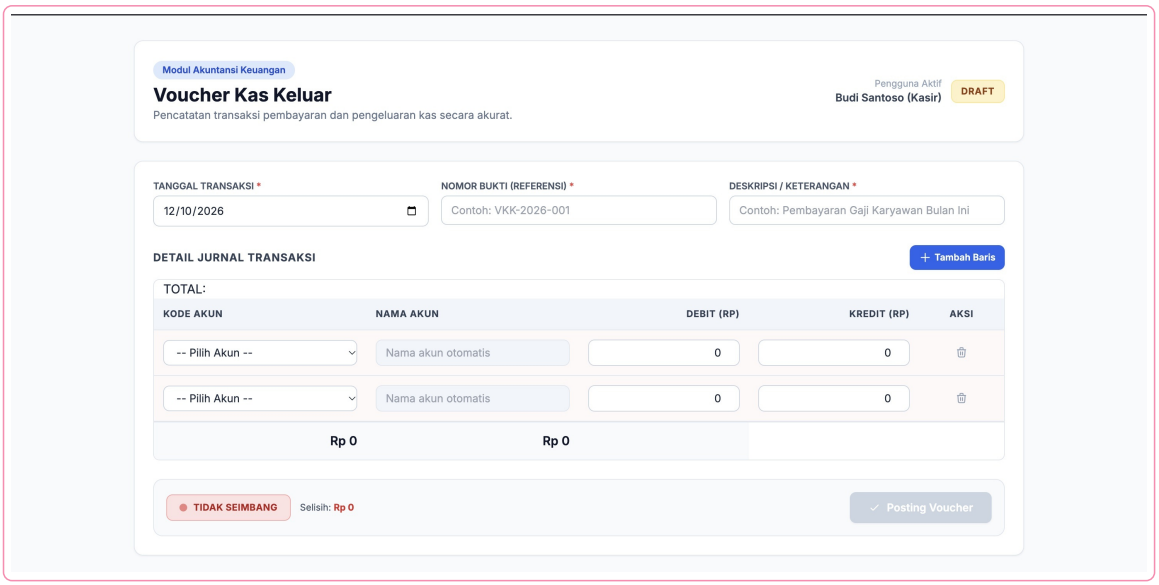
3.1 Web Form Prototipe Voucher Kas Keluar

Pengembangan prototipe formulir web untuk Voucher Kas Keluar dilakukan dengan mengimplementasikan tiga fitur teknis utama:

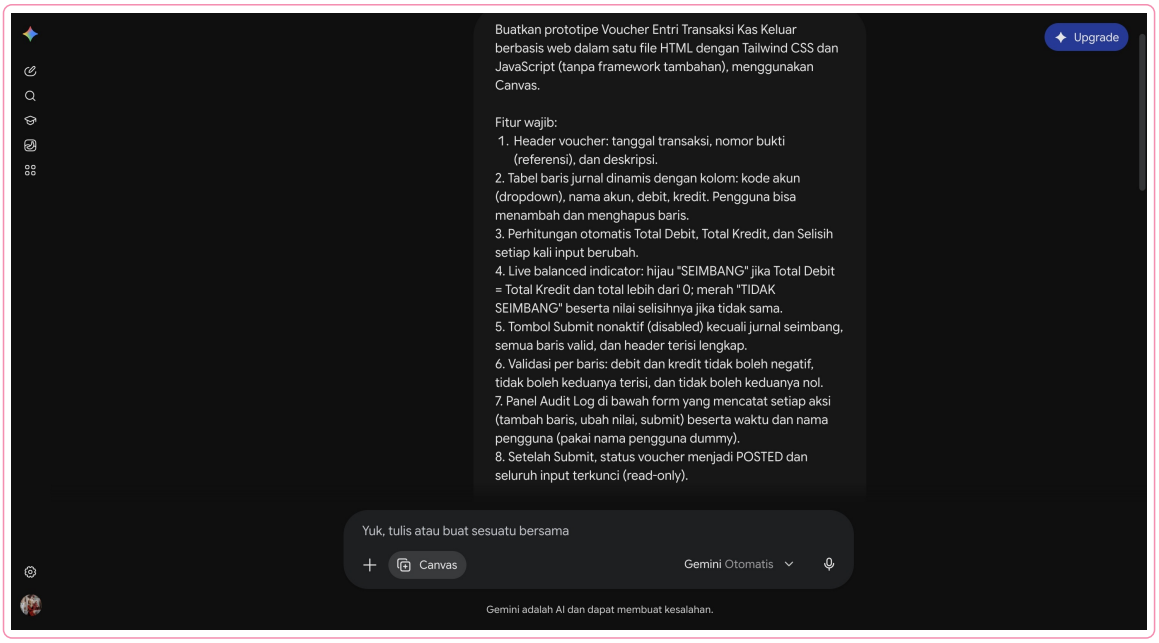
1. **Format Angka Rupiah Indonesia:** Menggunakan API `Intl.NumberFormat('id-ID', { style: 'currency', currency: 'IDR' })` sehingga menghasilkan format baku seperti `Rp 1.000.000,00`.
2. **Pencegahan Error Pembulatan Desimal:** Kalkulasi penjumlahan total debit dan kredit dilakukan dengan mengonversi seluruh masukan menjadi **satuan sen/integer** (`Math.round(nominal * 100)`) guna mengeliminasi *floating-point precision error* JavaScript.
3. **Tombol Jurnal Pembalik Kondisional:** Tombol *"Buat Jurnal Pembalik"* disembunyikan secara default dan hanya muncul saat voucher berstatus `POSTED`. Saat diklik, sistem otomatis membuat draf voucher baru dengan membalik posisi debit dan kredit serta mencantumkan referensi voucher asal.



Gambar 13. Antarmuka Formulir Web Voucher Kas Keluar Interaktif



Gambar 14. Pengujian Verifikasi Keseimbangan Debit dan Kredit pada Tabel Rincian



Gambar 15. Lembar Spesifikasi Prompt dan Log Aktivitas Audit Trail Web Form

3.2 Evaluasi Kepatuhan terhadap Standar Akuntansi & PSAK

Berdasarkan pengujian terhadap ketentuan PSAK 2 / ISAK 35 mengenai pengakuan kas dan setara kas, berikut adalah hasil evaluasi kepatuhan aturan bisnis logika form:

Tabel Ringkasan Verifikasi Aturan Bisnis (7 Butir)

No	Aturan Bisnis (Relevan dengan Standar Akuntansi)	Status Pemenuhan pada Form Web
1	Asas Berpasangan (Zero-Discrepancy): Total Debit = Total Kredit	**DIPENUHI** (Submit dikunci jika Debit != Kredit)
2	Arah Mutasi Kas Keluar: Debit Beban/Utang, Kredit Kas	**DIPENUHI** (Posisi akun sesuai standar)
3	Immutability Transaksi POSTED: Dilarang edit/hapus langsung	**DIPENUHI** (Form terkunci total saat POSTED)
4	Koreksi via Jurnal Pembalik (Storno Entry)	**DIPENUHI** (Tersedia tombol pembalik otomatis)
5	Validasi Saldo Kas Cukup (Anti-Negative Cash)	**BELUM DIPENUHI** (Butuh cek saldo backend)
6	Pisah Batas Periode Akuntansi (Cut-Off Period)	**BELUM DIPENUHI** (Butuh validasi periode buku)
7	Kelengkapan Jejak Audit (Audit Trail & Hash Checksum)	**DIPENUHI PADA BACKEND SQL**

3.3 Supabase Backend SQL: Security, Audit Trail, & Tamper Detection

Berikut adalah skrip backend SQL lengkap untuk Supabase (PostgreSQL) yang mengimplementasikan penguncian voucher `POSTED`, pencatatan `created_by` (via `auth.uid()`), `created_at`, RPC posting, RPC jurnal pembalik, serta perhitungan Hash SHA-256 untuk *tamper detection*.

```
-- 1. PENAMBAHAN KOLOM AUDIT & EXTENSION PGCRYPTO
CREATE EXTENSION IF NOT EXISTS pgcrypto;

ALTER TABLE journal_headers
    ADD COLUMN IF NOT EXISTS created_by UUID DEFAULT auth.uid(),
    ADD COLUMN IF NOT EXISTS created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    ADD COLUMN IF NOT EXISTS hash_checksum VARCHAR(64),
    ADD COLUMN IF NOT EXISTS reversal_of BIGINT REFERENCES journal_headers(id) ON DELETE RESTRICT;

-- 2. FUNGSI HITUNG HASH SHA-256
CREATE OR REPLACE FUNCTION calculate_journal_hash(p_header_id BIGINT)
RETURNS VARCHAR(64) AS $$
DECLARE
    v_ref_num VARCHAR(50); v_trx_date DATE; v_lines TEXT; v_payload TEXT;
BEGIN
    SELECT nomor_referensi, tanggal_transaksi INTO v_ref_num, v_trx_date FROM journal_headers WHERE id =
p_header_id;
    IF NOT FOUND THEN RETURN NULL; END IF;

    SELECT STRING_AGG(account_id || ':' || debit || ':' || credit, '|' ORDER BY id)
    INTO v_lines FROM journal_lines WHERE header_id = p_header_id;

    v_payload := v_ref_num || '|' || TO_CHAR(v_trx_date, 'YYYY-MM-DD') || '|' || COALESCE(v_lines, '');
    RETURN encode(digest(v_payload, 'sha256'), 'hex');
END;
$$ LANGUAGE plpgsql STABLE;

-- 3. TRIGGER PENGUNCI HEADER & LINES BERSTATUS POSTED
CREATE OR REPLACE FUNCTION trg_prevent_posted_header_modification() RETURNS TRIGGER AS $$
BEGIN
    IF OLD.status_posting = 'POSTED' THEN
        RAISE EXCEPTION 'Akses Ditolak: Jurnal ID % berstatus POSTED dan tidak dapat diubah atau dihapus!',
OLD.id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_lock_posted_header BEFORE UPDATE OR DELETE ON journal_headers
FOR EACH ROW EXECUTE FUNCTION trg_prevent_posted_header_modification();

-- 4. FUNGSI RPC POSTING
CREATE OR REPLACE FUNCTION post_journal(p_header_id BIGINT) RETURNS JSONB AS $$
DECLARE
    v_status VARCHAR(20); v_total_debit NUMERIC(15,2); v_total_credit NUMERIC(15,2); v_hash VARCHAR(64);
```

```
BEGIN
    SELECT status_posting INTO v_status FROM journal_headers WHERE id = p_header_id;
    IF v_status <> 'DRAFT' THEN RAISE EXCEPTION 'Hanya DRAFT yang dapat diposting.'; END IF;

    SELECT COALESCE(SUM(debit), 0.00), COALESCE(SUM(credit), 0.00)
    INTO v_total_debit, v_total_credit FROM journal_lines WHERE header_id = p_header_id;

    IF v_total_debit <> v_total_credit OR v_total_debit = 0 THEN
        RAISE EXCEPTION 'Total Debit dan Kredit harus seimbang dan > 0.';
    END IF;

    v_hash := calculate_journal_hash(p_header_id);
    UPDATE journal_headers SET status_posting = 'POSTED', hash_checksum = v_hash WHERE id = p_header_id;

    RETURN jsonb_build_object('success', true, 'message', 'Diposting!', 'hash_checksum', v_hash);
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- 5. FUNGSI RPC JURNAL PEMBALIK (STORNO)
CREATE OR REPLACE FUNCTION reverse_journal(p_header_id BIGINT) RETURNS JSONB AS $$
DECLARE
    v_orig RECORD; v_new_id BIGINT; v_new_ref VARCHAR(50);
BEGIN
    SELECT * INTO v_orig FROM journal_headers WHERE id = p_header_id;
    IF v_orig.status_posting <> 'POSTED' THEN RAISE EXCEPTION 'Hanya POSTED yang dapat dibalik.'; END IF;

    v_new_ref := 'REV-' || v_orig.nomor_referensi;
    INSERT INTO journal_headers (tanggal_transaksi, nomor_referensi, deskripsi, status_posting, created_by,
reversal_of)
    VALUES (CURRENT_DATE, v_new_ref, '[PEMBALIK] Atas ' || v_orig.nomor_referensi, 'DRAFT', auth.uid(),
p_header_id)
    RETURNING id INTO v_new_id;

    INSERT INTO journal_lines (header_id, account_id, debit, credit)
    SELECT v_new_id, account_id, credit AS debit, debit AS credit FROM journal_lines WHERE header_id =
p_header_id;

    PERFORM post_journal(v_new_id);
    RETURN jsonb_build_object('success', true, 'reversal_header_id', v_new_id);
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- 6. FUNGSI VERIFIKASI INTEGRITAS TAMPER DETECTION
CREATE OR REPLACE FUNCTION verify_journal_integrity(p_header_id BIGINT) RETURNS JSONB AS $$
DECLARE
    v_stored VARCHAR(64); v_computed VARCHAR(64); v_is_valid BOOLEAN;
BEGIN
    SELECT hash_checksum INTO v_stored FROM journal_headers WHERE id = p_header_id;
    v_computed := calculate_journal_hash(p_header_id);
    v_is_valid := (v_stored = v_computed);
    RETURN jsonb_build_object('header_id', p_header_id, 'is_valid', v_is_valid,
        'status_message', CASE WHEN v_is_valid THEN 'INTEGRITAS VALID' ELSE 'PERINGATAN: DATA TELAH
DIMANIPULASI!' END);
END;
$$ LANGUAGE plpgsql STABLE;
```

Nilai Pembuktian Bagi Auditor Finansial:

Penerapan Hash Checksum SHA-256 memberikan bukti perlindungan dua lapis (*two-factor immutability*). Meskipun seorang Administrator Basis Data (DBA) menonaktifkan trigger pengunci dan mengubah nominal di tabel secara langsung, nilai `computed_hash` yang dihitung ulang oleh auditor tidak akan pernah cocok lagi dengan `stored_hash`. Fungsi `verify_journal_integrity()` secara instan mendeteksi fraud tersebut.

BAB 4: PENUTUP DAN LAMPIRAN REPOSITORI PROJEK

4.1 Daftar Tautan Repositori & Ilustrasi Foto Website

Seluruh berkas kode sumber, skrip DDL/DML PostgreSQL, fungsi trigger PL/pgSQL, prototipe web form voucher, serta arsitektur backend Supabase untuk tugas ujian ini telah diunggah dan didokumentasikan secara rinci pada tautan repositori resmi di bawah ini:

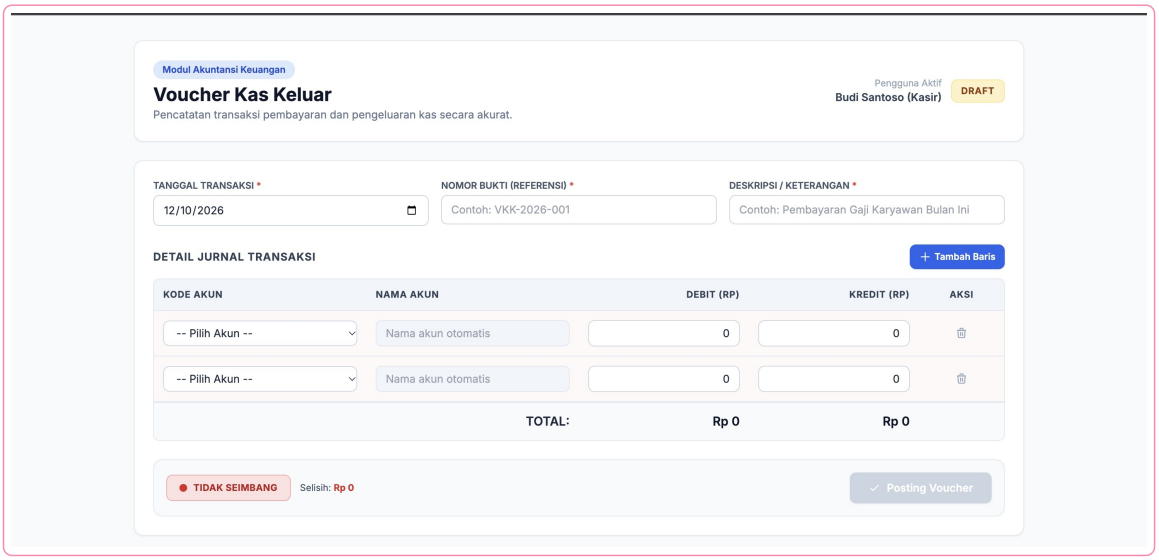
Tautan Resmi Repositori Projek:

1. **Link UTS (Pengkodean & Pemrograman Akuntansi):**
https://github.com/sabrinaimutbgt/UTS-PENGKODEAN-DAN-PEMOGRAMAN-SABRINA-SYIFA

2. **Link Repositori (AKSARA - Sistem Akuntabilitas Realisasi Anggaran):**
https://github.com/sabrinaimutbgt/AKSARA-Sistem-Akuntabilitas-Realisasi-Anggaran

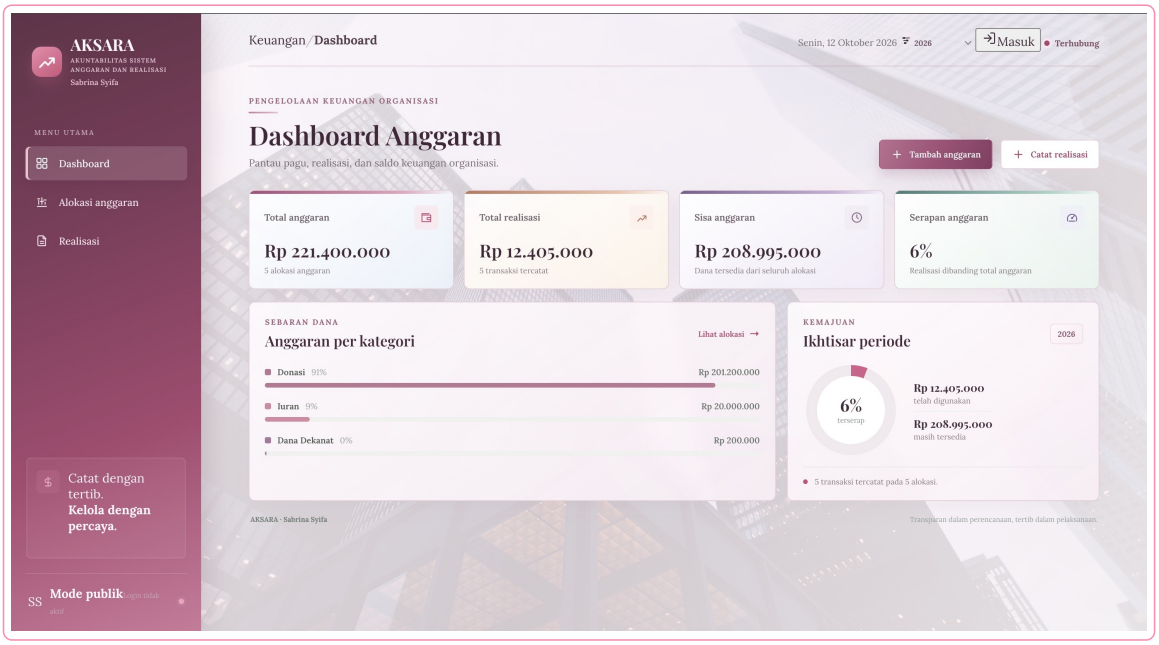
Visual Ilustrasi Foto Website Repositori & Platform Aplikasi:

• Ilustrasi Website Repositori Tugas UTS:



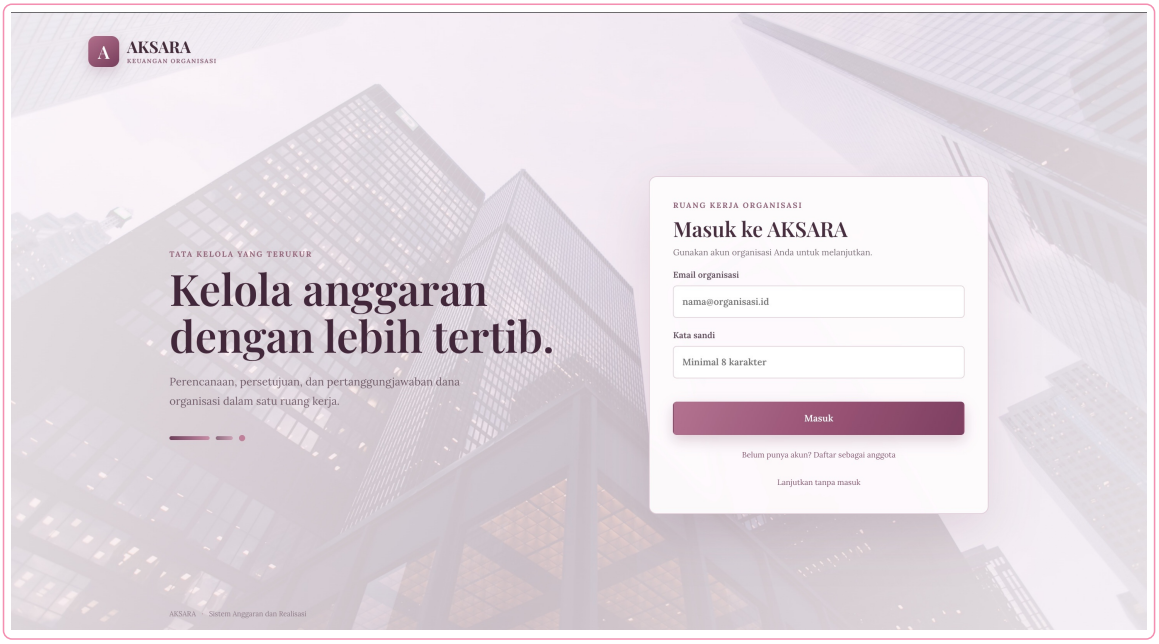
Gambar 16. Tampilan Antarmuka Repositori GitHub Tugas UTS Akuntansi

• Ilustrasi Antarmuka Aplikasi Web Akuntansi UTS:



Gambar 17. Tampilan Antarmuka Aplikasi Web Akuntansi & Form Interaktif

• Ilustrasi Website Repositori AKSARA (Sistem Akuntabilitas Realisasi Anggaran):



Gambar 18. Tampilan Antarmuka Repositori & Platform Sistem AKSARA FEB UNDIP

4.2 Log Prompt Kolaborasi AI Generatif

Proses penyusunan jawaban UTS ini dilakukan melalui kolaborasi terstruktur dengan AI Generatif. Berikut adalah rekapitulasi log prompt yang digunakan:

No	Fase / Topik Ujian	Tujuan Utama Prompt	Luaran / Hasil Teknis Utama
1	Soal 1: Arsitektur Data	Merancang skema relasional 3NF & ERD General Ledger Modul Ritel	Skema 3 tabel, penjelasan kardinalitas relasi, dan ERD visual.
2	Soal 1: DDL & Test	Membuat DDL re-runnable lengkap dengan PK, FK, & CHECK constraint	Sintaks DDL PostgreSQL, data dummy, dan bukti penolakan constraint error.
3	Soal 2: Trigger Inventory	Membuat trigger mutasi stok & anti-negative inventory PL/pgSQL	Fungsi trigger stok, pengujian 5 langkah, dan penjelasan AFTER INSERT.
4	Soal 2: Concurrency POS	Menganalisis race condition & penanganan persaingan checkout kasir	Tabel timeline race condition, kode SELECT FOR UPDATE & SERIALIZABLE.
5	Soal 3: Web Form	Membuat prototipe form voucher kas keluar interaktif & desimal sen	Kode HTML/JS form voucher kas keluar dengan proteksi floating-point error.
6	Soal 3: Supabase Backend	Mengunci voucher POSTED, RPC posting, RPC storno, & Hash SHA-256	Skrip backend Supabase SQL lengkap, trigger immutability, & tamper detection.